

OSDBMS Project Enhancement & Integration Strategy

Transforming Disaster Resource Allocation into an Exemplary OS & DBMS Engineering System

Project Title: Disaster Resource Allocation & Shelter Management	Course: 5th Sem OSDBMS PBL
Team ID: OSDBMS-V-2026-T135	Institution: Graphic Era (Deemed to be University)
Team Lead: Anushreya Tomar (24022202)	Team Members: Akanchha, Ishani, Simarjeet
Mentor: Dr. Ankit Tomar	Target Domain: Systems & DB Kernel Engineering

1. Executive Summary & Mentor Feedback Diagnosis

Core Feedback: Evaluators noted that the current project proposal leans heavily towards generic Web Application CRUD (Create, Read, Update, Delete) with insufficient demonstration of core Operating System and Database Management System engineering concepts.

Diagnostic Analysis: Standard web frameworks (like Node.js/Flask) hide OS concurrency and DB indexing under high-level abstractions. To impress evaluators and make this project worthy of capstones, publications, or hackathons, the team must make **OS Kernel principles** (Scheduling, IPC, Synchronization, Telemetry) and **DBMS Engine internals** (Spatial GIS Indexing, Pessimistic Locking, Active Triggers, Stored Procedures) **explicit, demonstrable core features**.

2. Deep Operating System (OS) Integration Strategy

A. Custom Priority Scheduler with Aging (OS CPU Scheduling → Evacuee Triage)

- **OS Concept:** Multi-Level Priority Queue (MLPQ) Preemptive Scheduling with Aging mechanism.
- **Disaster Context:** Treat evacuee requests as *OS Processes* and shelter beds as *CPU Time Slots*. Urgent medical cases (Priority 0) preempt general requests (Priority 2).
- **Aging Mechanism:** Implement an OS timer daemon. If a Priority 2 request waits past threshold *T*, its priority is dynamically bumped to prevent infinite starvation under severe disaster surges.
- **Evaluator Demo:** Show a live OS Process State diagram (READY, RUNNING, PREEMPTED, AGED).

B. Thread Synchronization & Deadlock Avoidance (OS Concurrency)

- **Producer-Consumer Buffer:** Field updates and IoT flood sensor streams produce events into a thread-safe Bounded Buffer with POSIX/Python Mutexes & Semaphores.
- **Banker's Algorithm for Multi-Resource Bundles:** When an evacuee needs a bundle (1 Bed + 1 Oxygen Cylinder + 1 Transport Unit), run Dijkstra's Banker's Algorithm to verify system state safety before allocation, preventing deadlocks between shelters.

C. Inter-Process Communication (IPC) & Shared Memory

- **OS Concept:** UNIX Domain Sockets & Shared Memory (shmget/mmap) with Semaphores.
- **Implementation:** Decouple high-throughput SOS intake from DB disk sync via Shared Memory queues for sub-millisecond intake during emergency spikes.

D. Real-Time OS Telemetry & System Load Simulator

- **Metrics Panel:** Display live CPU utilization, context switch counts, thread lock contention rates, and memory paging stats during a 1,000 requests/sec load test.

3. Advanced Database Management System (DBMS) Integration Strategy

A. Spatial Indexing & Geospatial Queries (GIS / R-Trees)

- **DBMS Concept:** Spatial R-Tree Indexing and K-Nearest Neighbor (KNN) spatial operators.
- **Implementation:** Store shelters and evacuees as POINT(lat, long) coordinates. Query nearest available shelters using spatial indices (e.g. ST_Distance_Sphere or PostGIS <-> operator) instead of application-level O(N) loops.

B. Pessimistic Concurrency Control & Isolation Levels

- **DBMS Concept:** Two-Phase Locking (2PL), Explicit Row Locks (SELECT FOR UPDATE), and ACID Transaction Isolation Levels.
- **Race Condition Proof:** Run 100 simultaneous threads attempting to book the final 2 beds. Show that standard queries cause overbooking (negative capacity), whereas pessimistic FOR UPDATE locking guarantees exact zero-overbooking integrity.

C. Active Database Triggers & Stored Procedures

- **Capacity Trigger (90% Warning):** DB trigger fires AFTER UPDATE on shelter occupancy. When capacity $\geq 90\%$, it automatically writes an event to alert_queue.
- **Atomic Stored Procedure (sp_AllocateShelterWithFallback):** Encapsulates multi-table operations (checking capacity, locking row, updating stock, inserting log) inside a single DB engine execution unit.

4. India-Based Datasets for Realistic Simulation

To ground your project in actual Indian disaster management conditions, incorporate the following authoritative real-world datasets:

Dataset Name & Source	Data Type & Description	OSDBMS Project Application
Kerala Floods 2018 (Kaggle / Kerala SDMA)	District-wise flood levels, affected population, relief camp counts, casualties.	Use as historical workload replay data to simulate district disaster spikes and benchmark OS scheduler under stress.
India Disaster Resource Network (IDRN) (NIDM / MHA)	National inventory of emergency equipment, specialized shelters, hospital beds across Indian districts.	Populate shelter and resource inventory schema with realistic Indian municipal logistics capacity.
OpenStreetMap / HDX India Emergency Infrastructure	Geospatial shapefiles of schools, community halls, stadiums used as emergency shelters in India.	Benchmark Spatial R-Tree Indexing and 5km KNN proximity queries on real Indian latitude/longitude coordinates.
State Shelter Board Data (DUSIB / NULM / SDMAs)	Geocoded shelter list with capacities, drinking water, medical amenities, and gender accommodations.	Test multi-criteria DBMS filtering and Banker's Algorithm multi-resource allocation bundles.

5. Innovative Extra Features to Stand Out

Adding these high-impact features will make your project stand out in evaluations, hackathons, and research paper presentations:

1. Offline-First SMS/LoRa Gateway (Low-Level OS IPC Serial Daemon)

In major Indian floods (e.g. Chennai, Assam), cellular internet fails first! Build a low-level serial socket daemon in the OS backend that receives short SMS / USSD strings or LoRa mesh packets from field workers and parses them directly into the DBMS.

2. Multi-Lingual Voice Triage Intake (Vernacular Voice-to-SQL)

Integrate speech-to-text (e.g. AI4Bharat Bhashini API) so non-literate evacuees or field workers can speak in Hindi, Assamese, or Malayalam. The backend translates spoken audio into structured OS triage priority flags.

3. AI/ML Flood Inundation & Capacity Predictor (Python Scikit-Learn + DBMS)

Train a lightweight ML model (XGBoost/RandomForest) on IMD rainfall data to forecast shelter capacity exhaustion 6 hours in advance, triggering pro-active DB reallocation stored procedures.

4. Tamper-Proof Aid Audit Trail (Cryptographic Hash Chains in DBMS)

To prevent aid diversion during relief operations, hash each supply allocation transaction (SHA-256 block chain) inside the database. Evaluators will appreciate this lightweight ledger implementation.

5. Live GIS Map & Heatmap Dashboard (OpenStreetMap / ISRO Bhuvan Integration)

Display live spatial heatmaps of shelter occupancy rates and active evacuation buffer zones directly linked to spatial SQL queries.

6. Team Action Plan & Role Distribution

Team Member	Core Responsibility	Specific Deliverables & Technical Tasks
Anushreya Tomar (Team Lead & DB)	Database Engine & Concurrency Locks	• Implement Spatial R-Tree Indexing in MySQL/PostGIS. • Write SELECT FOR UPDATE Pessimistic Transaction locks. • Build DB 90% alert triggers and atomic stored procedures.
Akanchha Singh (Frontend & GIS)	Interactive UI & GIS Heatmaps	• Build Leaflet.js / OpenStreetMap GIS dashboard. • Add live OS Telemetry Panel (mutex wait times, context switches). • Integrate India datasets (Kerala 2018 / IDRN).
Ishani Nautiyal (OS Scheduler)	OS Scheduler & Triage Logic	• Code MLFQ Priority Queue with Aging daemon. • Implement Dijkstra's Banker's Algorithm for resource bundles. • Build process state transition visualizer.
Simarjeet Kaur (IPC & Testing)	IPC Engine & Benchmark Suite	• Build Producer-Consumer Bounded Buffer with semaphores. • Develop 1,000 concurrent request load tester. • Prepare performance benchmark graphs for presentation.

Conclusion & Roadmap

By implementing these OS kernel mechanisms, DBMS engine optimizations, and real Indian datasets, Team OSDBMS-V-2026-T135 will transform this project into an exemplary engineering milestone. This plan ensures maximum marks from evaluators and provides a solid foundation for hackathons and research publications.